

Soluzioni alla simulazione d'esame del 17.01.2025 (PARTE-B)

Esercizio 4: Si consideri l'equazione di ricorrenze

$$T(n) = 6T\left(\frac{n}{b}\right) + \Theta(n \log n). \quad (1)$$

Si risolva (1) al variare di $b > 1$, utilizzando il metodo Master.

Watershed function: $w(n) = n^{\log_b 6}$;

Driving function: $f(n) = \Theta(n \log n)$.

La relazione tra la driving function e la watershed function dipende dalla relazione tra n e $n^{\log_b 6}$.

$$n \leq n^{\log_b 6} \iff 1 \leq \log_b 6 \iff b \leq 6.$$

Abbiamo quindi tre casi.

- Se $b < 6$, ovvero $n < n^{\log_b 6}$, $\exists \varepsilon : 0 < \varepsilon < \log_b 6 - 1$ tale che $f(n) = O(n^{\log_b 6 - \varepsilon})$, quindi ci troviamo nel primo caso del The Master e $T(n) = \Theta(n^{\log_b 6})$.
- Se $b = 6$, ovvero $n = n^{\log_b 6}$, $\exists k \geq 0, k = 1$ tale che $f(n) = \Theta(n^{\log_b 6} \log^k n) = \Theta(n \log n)$, quindi ci troviamo nel secondo caso del The Master e $T(n) = \Theta(n \log^2 n)$.
- Se $b > 6$, ovvero $n > n^{\log_b 6}$, $\exists \varepsilon : 0 < \varepsilon < 1 - \log_b 6$ tale che $f(n) = \Omega(n^{\log_b 6 + \varepsilon})$, inoltre vale la condizione di regolarità:
 $\exists c < 1 : a f(\frac{n}{b}) \leq c f(n) \Leftrightarrow 6 \frac{n}{b} \log(\frac{n}{b}) \leq c n \log n$ *asintoticamente*
 $\Leftrightarrow 6 \frac{n}{b} \log n - 6 \frac{n}{b} \log 6 \leq c n \log n$ *asintoticamente* $\Leftrightarrow$
 $\Leftrightarrow \frac{6}{b} n \log n \leq c n \log n \Leftrightarrow c \geq \frac{6}{b}$ quindi la condizione è equivalente all'esistenza di $c : \frac{6}{b} \leq c < 1$, che è vero dato che $b > 6$. Quindi, essendo nel terzo caso del The Master avremo che $T(n) = \Theta(n \log n)$.

Si stabilisca per quali valori di b la soluzione $T(n)$ a (1) soddisfa:

(i) $T(n) = O(n^2)$, (ii) $T(n) = \Theta(n \log n)$, (iii) $T(n) = \omega(n)$.

La soluzione a (1) è $T(n) = \begin{cases} \Theta(n^{\log_b 6}) & b < 6 \\ \Theta(n \log^2 n) & b = 6 \\ \Theta(n \log n) & b > 6 \end{cases}$

(i) $c n^{\log_b 6} = O(n^2)$ solo se $\log_b 6 \leq 2 \Leftrightarrow b^2 \geq 6 \Leftrightarrow b \geq \sqrt{6}, b < 6$
e $c n \log^2 n = O(n^2)$ e $c n \log n = O(n^2)$.
Quindi $T(n) = O(n^2)$ per $b \geq \sqrt{6}$.

(ii) $c n^{\log_b 6} \neq \Theta(n \log n)$, $c n \log^2 n \neq \Theta(n \log^2 n)$. Quindi $T(n) = \Theta(n \log n)$ solo per $b > 6$.

(iii) $c n^{\log_b 6} = \omega(n)$ perché $\log_b 6 > 1$ per $b < 6$. Quindi $T(n) = \omega(n), \forall b > 1$.

Esercizio 2: Si dimostra che il problema della codifica prefix-free gode delle proprietà di scelte greedy.

Dobbiamo mostrare che esiste una codifica prefix-free tale che nel corrispondente albero prefix-free i due caratteri meno frequenti abbiano profondità massima.

Per mostrarlo consideriamo un albero di codifica prefix-free T e assumiamo che i caratteri meno frequenti x, y non abbiano massima profondità.

Siano invece a, b i caratteri che si trovano alla massima profondità di T .

Supponiamo $x.\text{freq} \leq y.\text{freq}$, $a.\text{freq} \leq b.\text{freq} \Rightarrow x \neq b$ e $y \neq a$.

Definiamo T' l'albero di codifica prefix-free ottenuto da T scambiando le posizioni di x e a e sia T'' l'albero di codifica prefix-free ottenuto da

T scambiando b con y. Allora T'' è un albero
 di codifica prefix-free in cui le foglie di profondità
 massima corrispondono a x e y. Inoltre, T'' è ottimo
 perché il suo costo non è più grande di quello di
 T, che è minimo. Infatti.

$$B(T) = \sum_{c \in C} c \cdot \text{freq} \cdot d_T(c), \quad B(T') = \sum_{c \in C} c \cdot \text{freq} \cdot d_{T'}(c), \quad \text{e}$$

$$B(T'') = \sum_{c \in C} c \cdot \text{freq} \cdot d_{T''}(c).$$

$$B(T') \leq B(T) \quad \text{perché} \quad B(T) - B(T') \geq 0, \quad \text{infatti}$$

$$B(T) - B(T') = \sum_{c \in C} c \cdot \text{freq} \cdot d_T(c) - \sum_{c \in C} c \cdot \text{freq} \cdot d_{T'}(c) =$$

$$= x \cdot \text{freq} \cdot d_T(x) + a \cdot \text{freq} \cdot d_T(a) - x \cdot \text{freq} \cdot d_{T'}(x) - a \cdot \text{freq} \cdot d_{T'}(a) =$$

$$= a \cdot \text{freq} (d_T(a) - d_{T'}(a)) - x \cdot \text{freq} (d_{T'}(x) - d_T(x)) =$$

$$= a \cdot \text{freq} (d_T(a) - d_T(x)) - x \cdot \text{freq} (d_T(a) - d_T(x)) =$$

$$= (a \cdot \underbrace{\text{freq}}_0 - x \cdot \underbrace{\text{freq}}_0) (\underbrace{d_\tau(e)}_0 - \underbrace{d_\tau(x)}_0) \geq 0.$$

Analogamente, $B(T'') \leq B(T') \leq B(T)$. $\square$

Esercizio 3: Dato un grafo $G = (V, E)$, $w: E \rightarrow \mathbb{R}$ e $V = \{1, \dots, n\}$ $l_{ij}^{(r)}$ denota il peso di un cammino minimo da i a j contenuto al più a r archi. Si scrive l'espressione ricorsiva di $l_{ij}^{(r)}$.

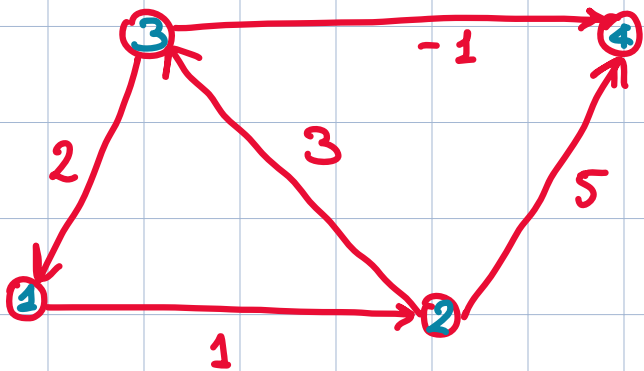
$$l_{ij}^{(0)} = \begin{cases} 0 & i=j \\ \infty & i \neq j \end{cases}, \quad l_{ij}^{(r)} = \min_{1 \leq k \leq n} \{ l_{ik}^{(r-1)} + w_{kj} \}.$$

Si scrive il codice della procedura EXTEND-SHORT-EST-PATHS che calcola la matrice $L^{(r)} = (l_{ij}^{(r)})$.

EXTEND-SHORTEST-PATHS ($L^{(z-1)}, W, L^{(z)}, n$)

1. for $i = 1$ to n
2. for $j = 1$ to n
3. for $k = 1$ to n
4. $l_{ij}^{(z)} = \min \{ l_{ij}^{(z-1)}, l_{ik}^{(z-1)} + w_{kj} \}$

Infine, si usi l'algoritmo ALL-PAIRS-SHORTEST-PATHS per il grafo $G = (V, E)$ in figura.



$$W = \begin{matrix} L^{(1)} \\ \begin{pmatrix} 0 & 1 & \infty & \infty \\ \infty & 0 & 3 & 5 \\ 2 & \infty & 0 & -1 \\ \infty & \infty & \infty & 0 \end{pmatrix} \end{matrix}$$

ho usato
FASTER-APSP
ma va bene
pure usare SLOW-APSP

$$L^{(2)} = \begin{pmatrix} 0 & 1 & 3 & 5 \\ 5 & 0 & 3 & 2 \\ 2 & 2 & 0 & -1 \\ \infty & \infty & \infty & 0 \end{pmatrix}, L^{(4)} = \begin{pmatrix} 0 & 1 & 3 & 2 \\ 5 & 0 & 3 & 2 \\ 2 & 2 & 0 & -1 \\ \infty & \infty & \infty & 0 \end{pmatrix}$$

$L^{(4)} = L^{(3)} = L^{(n-1)} = L$. Quindi L è una soluzione al problema.